LPM_ROM 与 LPM_RAM 实验报告

目录

一、实验内容 2
二、实验原理 2
三、实验任务 3
四、实验步骤 3
五、软件编译 6
六 仿真波形图及其分析报告 6
七 实验结果及问题分析或结论 7
八 思考题 10

一、实验内容

（一）LPM_ROM 实验

- 了解 LPM_ROM 在 CPU 中的作用及基本原理。
- 学会使用 Quartus II 软件对 LPM_ROM 进行设置、编译和引脚锁定。
- 观察并验证 LPM_ROM 中存储的微程序数据在实验电路中的输出情况。

（二）LPM_RAM 实验

- 理解 LPM_RAM 作为数据存储器的功能和工作方式。
- 掌握在 Quartus II 中对 LPM_RAM 的调用、参数设置、初始化以及相关操作。
- 通过实验操作，实现对 LPM_RAM 的数据写入和读取，并对比验证数据的准确性。

二、实验原理

（一）LPM_ROM 原理

1. PM_ROM（只读存储器）是一种存储固定数据的存储器，在 CPU 中用于存储程序代码或常量数据。其特点是在正常工作时只能读取数据，不能写入数据。
2. 当输入地址信号时，ROM 内部的地址译码器会将地址信号转换为对应的存储单元选择信号，选中相应的存储单元，然后将存储单元中的数据输出到数据线上。地址线的位数决定了可寻址的存储单元数量，数据线的位数决定了每个存储单元中存储数据的宽度。例如，本实验中 6 位地址线（address [5..0]）可以寻址 64 个存储单元，每个单元存储 24 位数据（q [23..0]）。
3. ROM 的工作速度取决于其存储单元的物理结构和访问方式。在配置 LPM_ROM 时，选择合适的时钟模式（如单时钟模式）可以确保数据在正确的时钟沿被稳定输出，以满足系统对微程序读取的时序要求。

（二）LPM_RAM 原理

1. LPM_RAM（随机存取存储器）是一种可以随时读写数据的存储器，用于存储 CPU 运行过程中的数据。
2. 实验中的 LPM_RAM 数据宽度为 8 位（datal7..0] 和 q [7..0]），地址宽度为 8 位（address [7..0]），存储单元总数为 256 个字节。
3. 它具有读写控制信号（wren），当 wren 为高电平时进行写操作，将输入数据写入指定地址单元；当 wren 为低电平时进行读操作，将指定地址单元的数据输出。

三、实验任务

（一）LPM_ROM 任务

1. 完成 LPM_ROM 的创建和配置，确保其参数设置正确，如地址线宽度、数据线宽度、存储单元数量、时钟模式等。
2. 选择合适的初始化文件（.mif 文件）加载到 LPM_ROM 中，并正确设置相关参数加载到 LPM_ROM 中，实现 ROM 的初始化。
3. 对设计进行编译，锁定引脚，将设计下载到 FPGA 目标芯片中。
4. 观察实验电路中，通过按键输入地址，在数码管上显示 LPM_ROM 中对应地址的微指令数据，并与预期数据进行对比验证。

（二）LPM_RAM 任务

1. 完成 LPM_RAM 的创建和配置，确保其参数设置正确，如地址线宽度、数据线宽度、存储单元数量、时钟模式等。

2. 实现通过实验平台按键向 LPM_RAM 写入数据，并能够准确读出写入的数据，通过数码管显示结果，验证 RAM 的读写功能正常。

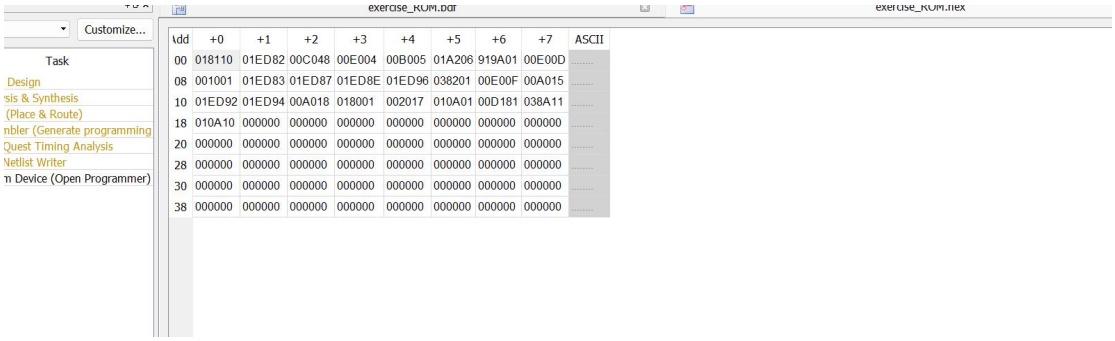
四、实验步骤

（一）LPM_ROM 实验步骤

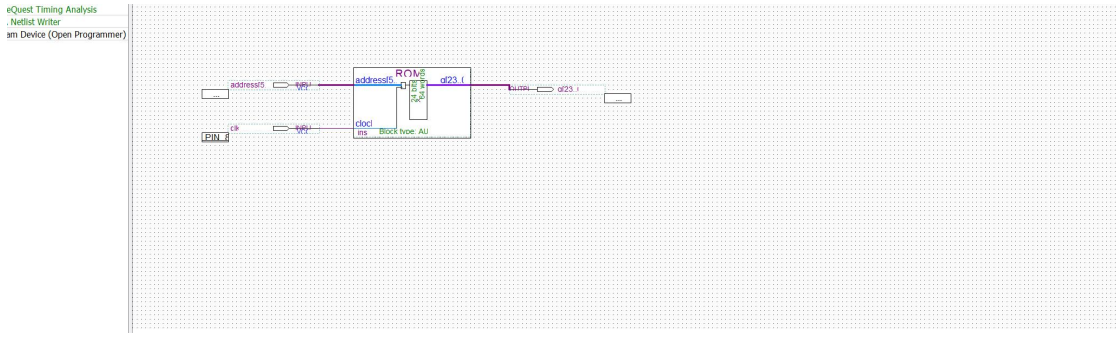
1. 工程创建与 LPM_ROM 生成

1. 新建工程，指定工程名称和保存路径。
2. 在工程中，通过 MegaWizard Plug-In Manager 创建 LPM_ROM 文件，选择 ROM: 1 - PORT 类型，设定输出文件名为 ROM0（自定义名称，避免与工程名重复）。
3. 在 MegaWizard Plug-In Manager 的设置页面中，进行如下操作：
4. 将数据输出位数（q）设置为 24 位，以适应微程序数据的宽度。
5. 设定数据存储单元总数为 64，根据地址线位数确定可寻址范围。
6. 选择时钟模式为单时钟（Single clock），保证数据读取的时序稳定。
7. 在“Mem Init”选项中，选择“是（Yes）”，点击“Browse...”按钮，选择微程序数据文件（如 rom_a.mif），该文件中的数据将用于初始化 ROM。同时，确保文件格式和内容符合 ROM 的配置要求，如地址和数据的位宽等。

2. 编写微程序数据文件



3. 完成实验电路结构图



4. 完成编译

5. 锁定引脚

根据实验平台的硬件连接要求，在 Quartus II 软件中进行引脚锁定操作。例如，将 ROM 的地址线（address [5..0]）分别与实验平台上用于输入地址的键 2/1 所连接的引脚（如 PIN_60、PIN_65 等）进行绑定；将时钟信号（inclock）与键 8 连接的引脚（PIN_84）绑定；将数据线（q [23..0]）与用于显示数据的数码管（数码 8/7/6/5/4/3）所连接的引脚（如 PIN_68、PIN_72 等）进行绑定。

6. 下载文件与测试

1. 将编译生成的文件下载到目标芯片中。

2. 在实验平台上，选择模式 0，通过操作键 2/1 输入不同的 6 位地址值，然后按下键 8 锁存地址，观察数码管上显示的 24 位微指令数据。将显示的数据与预期的微程序数据（如 rom_a.mif 文件中对应地址的数据）进行对比，验证 LPM_ROM 在不同地址下输出数据的准确性。

（二）LPM_RAM 实验步骤

2. 工程创建与 LPM_RAM 生成

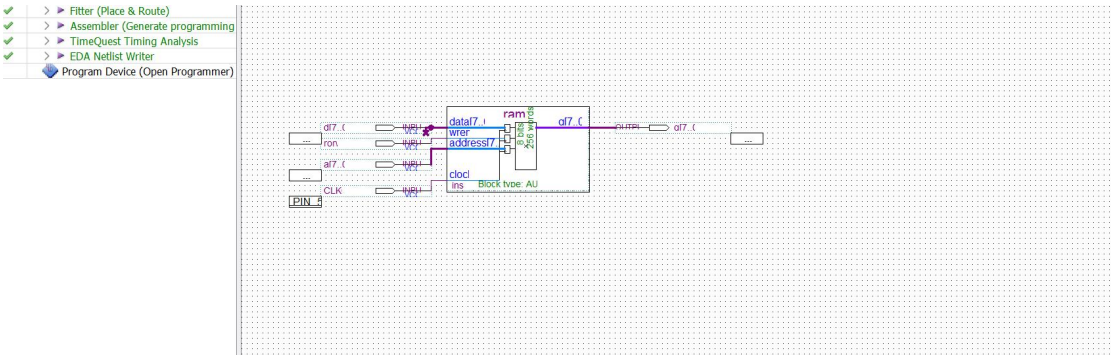
1. 新建工程，指定名称和路径。
2. 通过 MegaWizard Plug-In Manager 创建 LPM_RAM 文件

3. LPM_RAM 参数设置

1. 在 MegaWizard Plug-In Manager 的设置页面中，进行如下配置：
   1. 根据实验需求，确定端口注册方式，如本实验中对“data”和“wren”输入端口进行注册。
   2. 设置地址线宽度为 8 位（address [7..0]），数据线宽度为 8 位（data [7..0] 和 q [7..0]）
   3. 在“Mem Init”部分，选择指定初始内容文件，点击“Browse...”，选择准备好的 RAM 初始数据文件

4. 编写微程序数据文件

5. 完成实验电路结构图



6. 完成编译

7. 锁定引脚

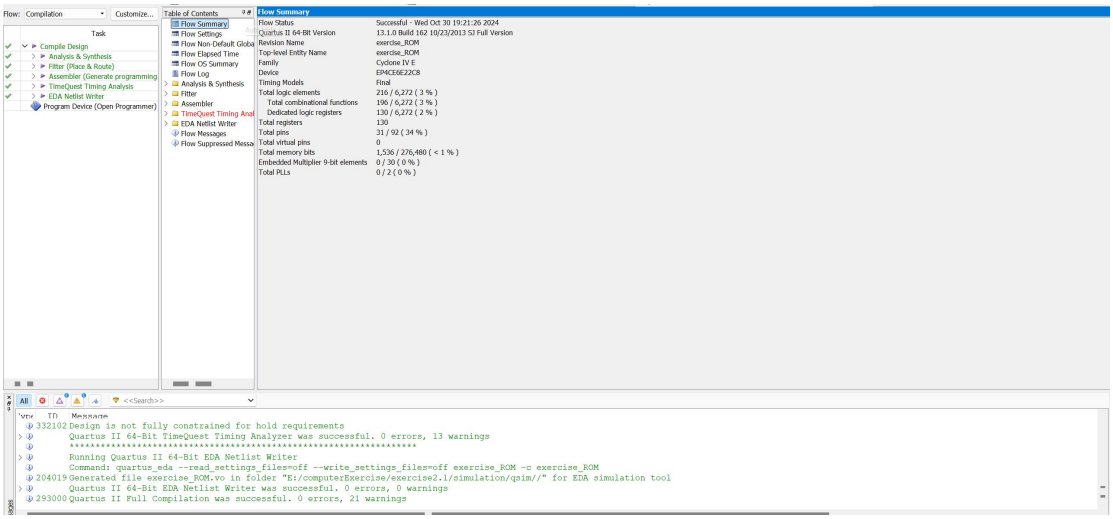
按照实验平台硬件连接，在 Quartus II 软件中锁定 RAM 的引脚。例如，将地址线（address [7..0]）与键 4/3 连接的引脚（如 PIN_60、PIN_65 等）绑定；将数据线（data [7..0]）与键 2/1 连接的引脚（用于输入数据）以及数码 8/7 显示数据的引脚绑定；将写使能信号（wren）与键 8（控制读写允许）连接的引脚绑定；将地址锁存信号与键 7 连接的引脚绑定。

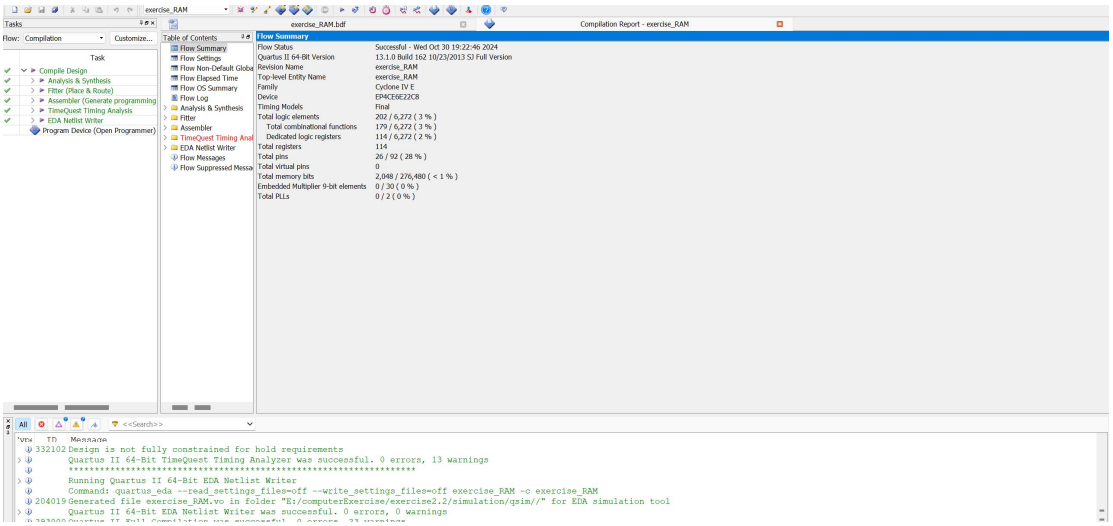
8. 读写操作测试

1. 初始状态下，将键 8 置“0”（读允许），通过键 4/3 输入 8 位地址，观察数码 8/7 上是否显示对应地址的初始数据（RAM 中有预加载数据）。
2. 然后，将键 8 置“1”（写允许），通过键 2/1 输入 8 位数据，按下键 7 锁存地址，完成数据写入操作。

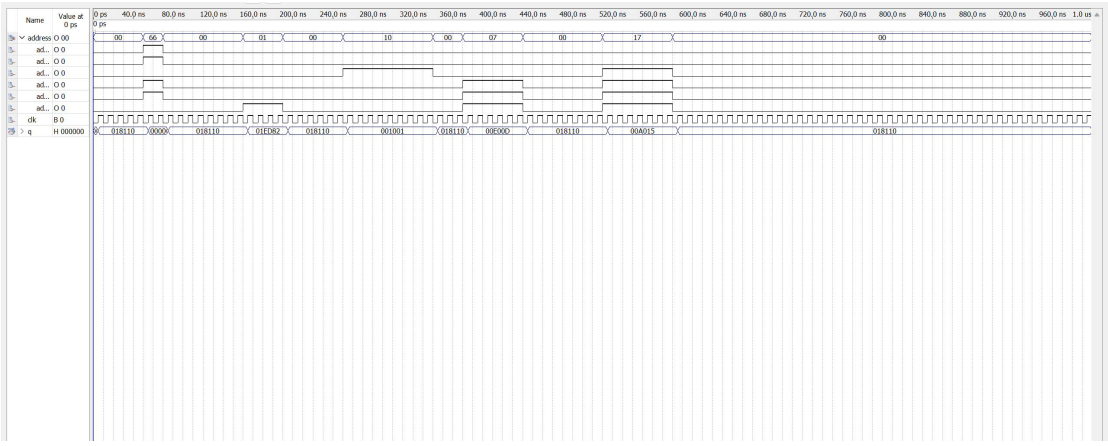
3. 再次将键 8 置“0”，通过键 4/3 输入刚才写入数据的地址，观察数码 8/7 上显示的数据是否与写入的数据一致，以此验证 LPM_RAM 的读写功能。

五、软件编译

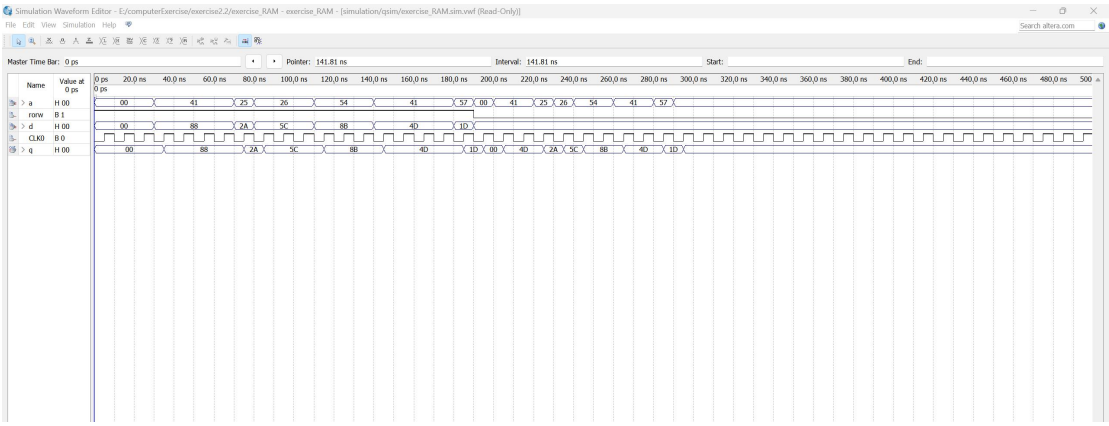




六 仿真波形图及其分析报告



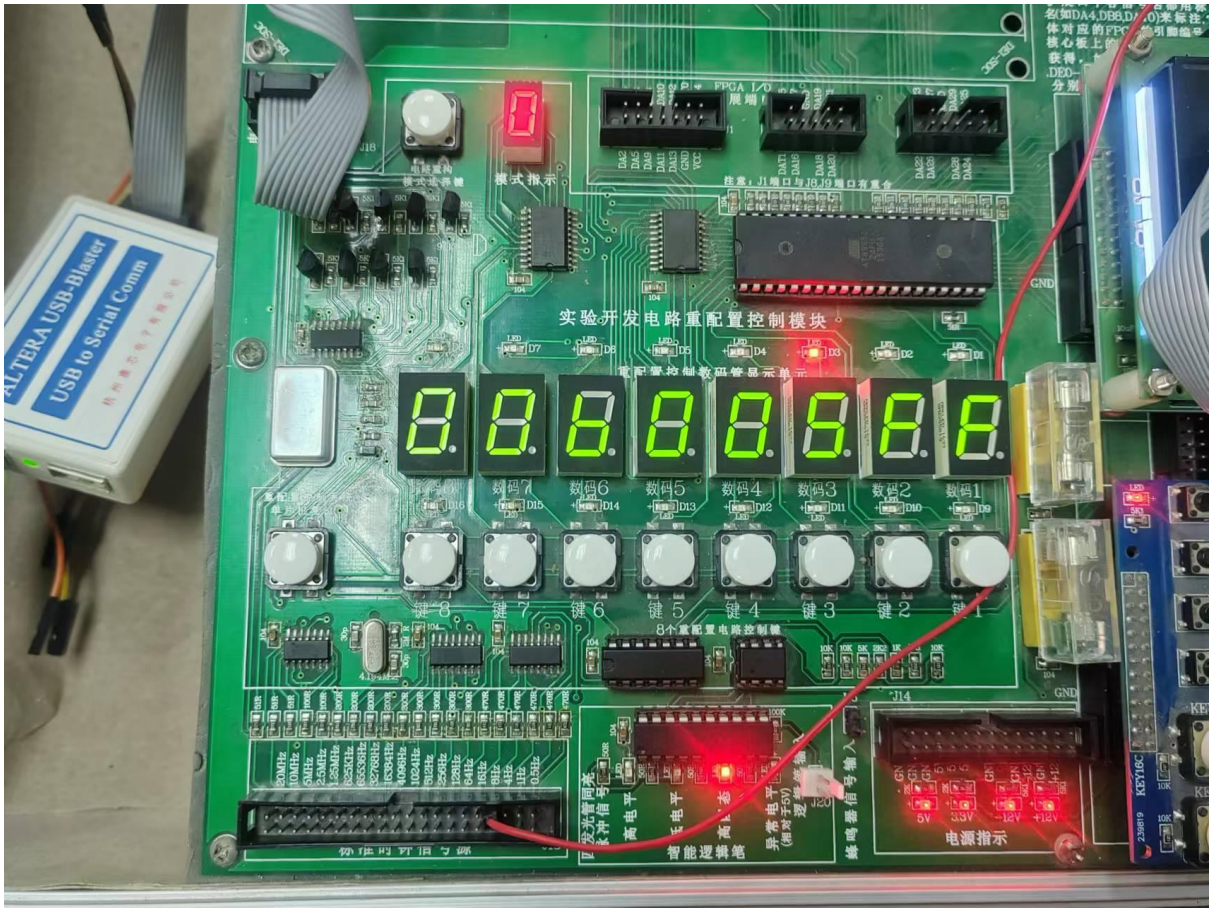
随着地址线信号的变化，在时钟信号的上升沿（假设 inclock 为上升沿有效），数据线（q）输出对应地址存储的微指令数据。例如，当地址为 00H 时，数据线输出 018110H ，与 中 rom_a.mif 文件中地址 00H 对应的微指令数据一致。测试其他地址，输出数据与储存数据也一致。

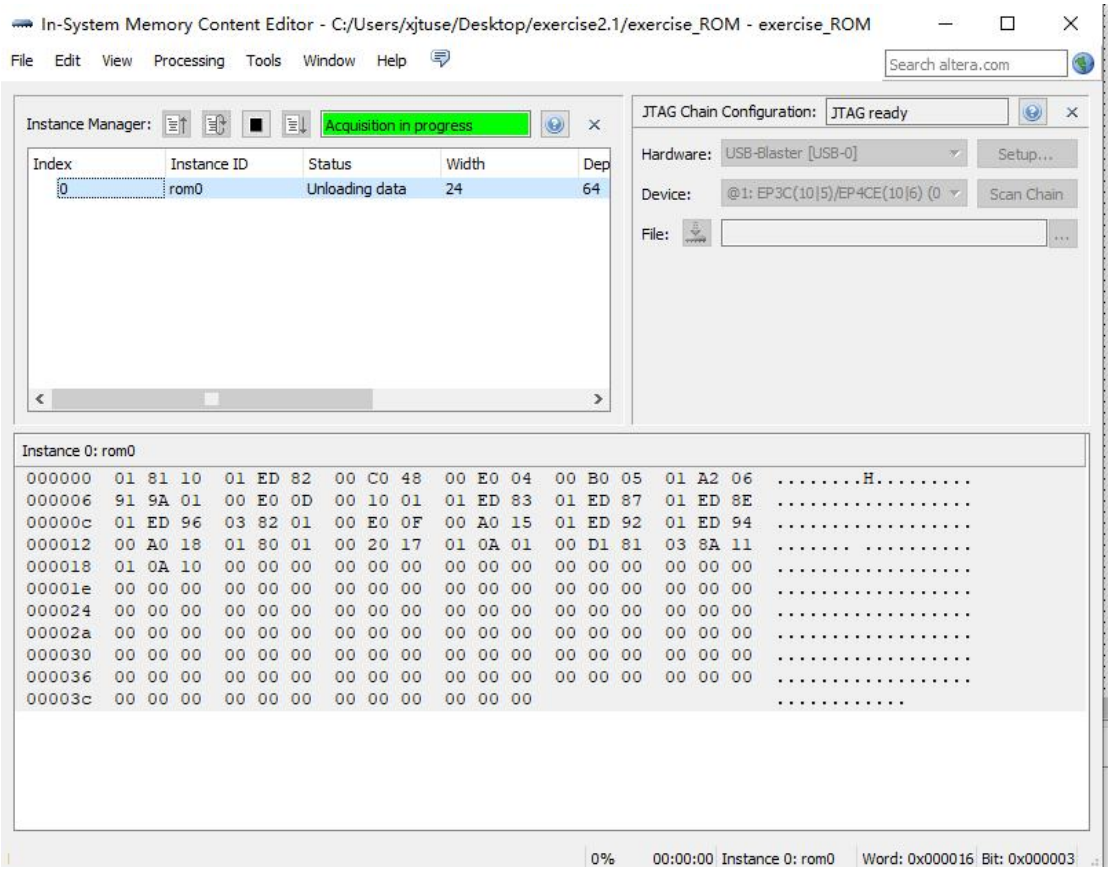


- 在写操作阶段（romw 为高电平），根据地址线和数据线的信号变化，可以看到数据在时钟上升沿被写入到指定地址单元，25H 写入 2AH
- 在读操作阶段（wren 为低电平），输入地址后，在时钟上升沿，数据线（q）应输出对应地址单元存储的数据，且与之前写入的数据一致。地址 25H，此时 wren 为低电平，数据线应输出 2AH，读操作正确。

七 实验结果及问题分析或结论

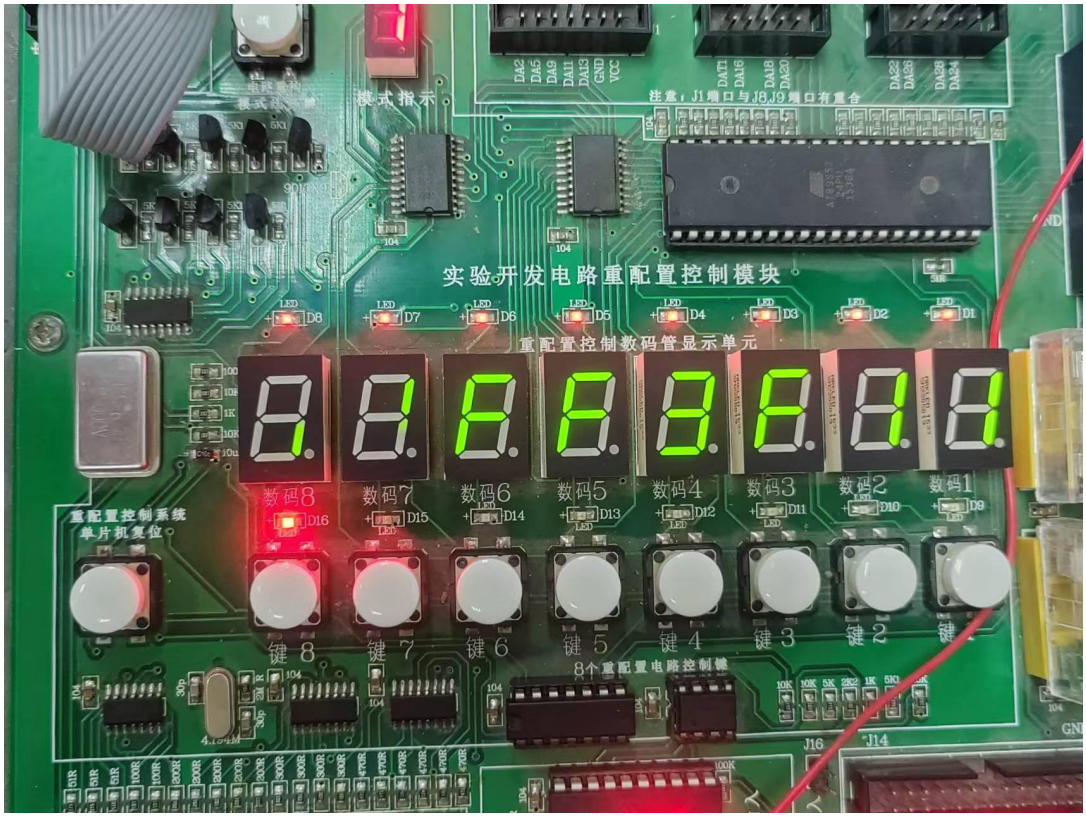
（1）

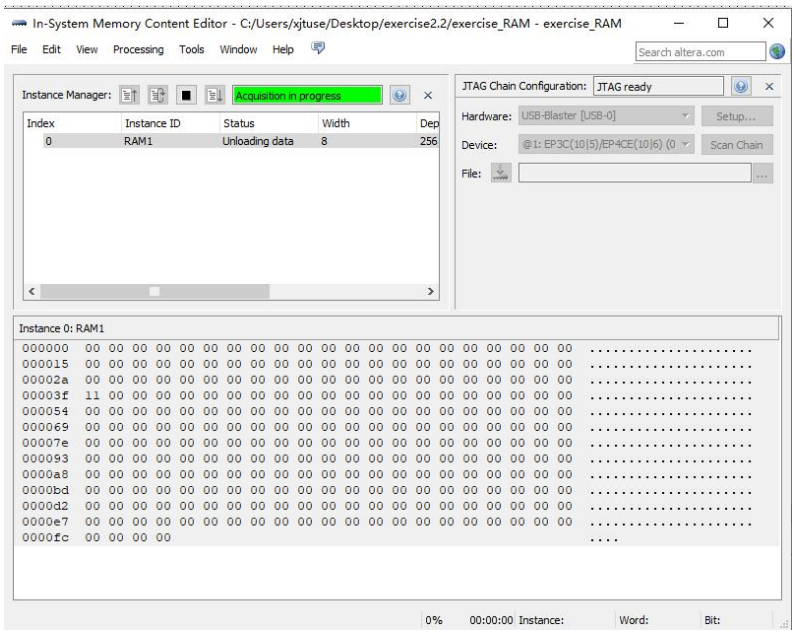

ALTERA USB-Blaster
USB to Serial Comm
实验开发电路重配置控制模块
模式指示
数码7
数码6
数码5
数码4
数码3
数码2
数码1
键8
键7
键6
键5
键4
键3
键2
8个重配置电路控制键
智能逻辑笔
电源指示


In-System Memory Content Editor - C:/Users/xjtuse/Desktop/exercise2.1/exercise_ROM - exercise_ROM
File Edit View Processing Tools Window Help
Search altera.com
Instance Manager:
Acquisition in progress
Index Instance ID Status Width Dep
0 rom0 Unloading data 24 64
JTAG Chain Configuration: JTAG ready
Hardware: USB-Blaster [USB-0]
Setup...
Device: @1: EP3C(10|5)/EP4CE(10|6) (0
Scan Chain
File:
Instance 0: rom0
000000 01 81 10 01 ED 82 00 C0 48 00 E0 04 00 B0 05 01 A2 06 .....H.....
000006 91 9A 01 00 E0 0D 00 10 01 01 ED 83 01 ED 87 01 ED 8E .....
00000c 01 ED 96 03 82 01 00 E0 0F 00 A0 15 01 ED 92 01 ED 94 .....
000012 00 A0 18 01 80 01 00 20 17 01 0A 01 00 D1 81 03 8A 11 ....
000018 01 0A 10 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00001e 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000024 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00002a 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000030 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
000036 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....
00003c 00 00 00 00 00 00 00 00 00 00 00 00 .....
0% 00:00:00 Instance 0: rom0 Word: 0x000016 Bit: 0x000003

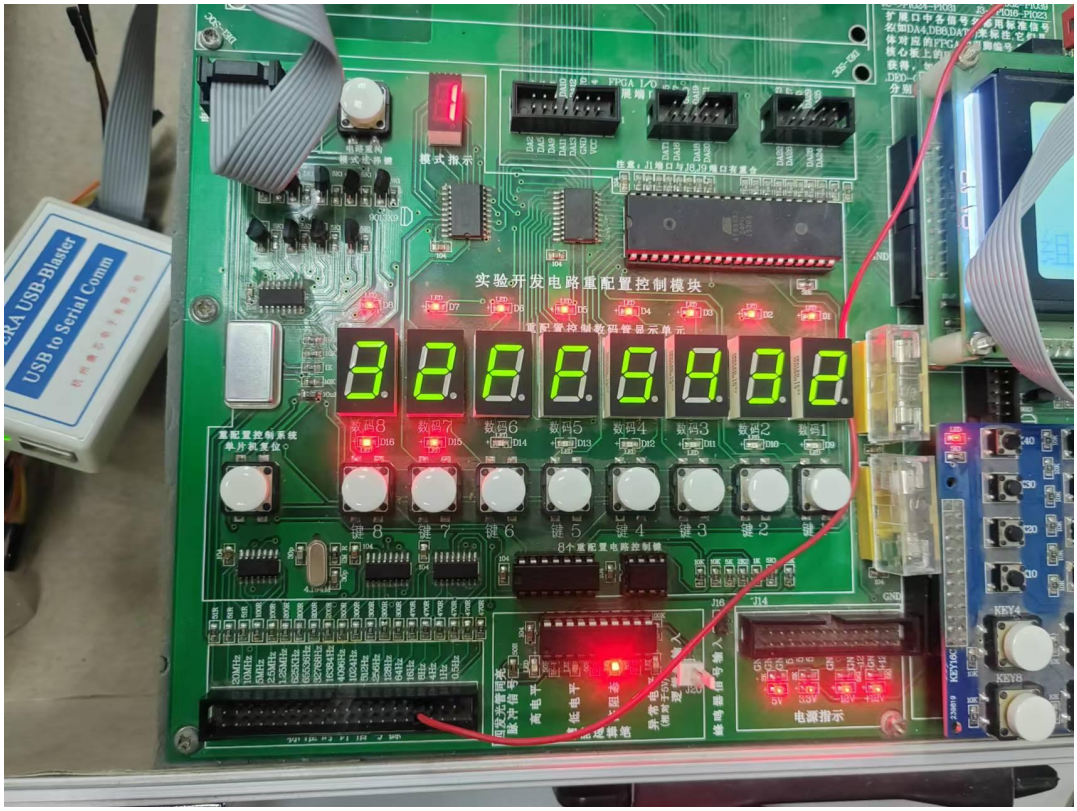
输出与预期一致

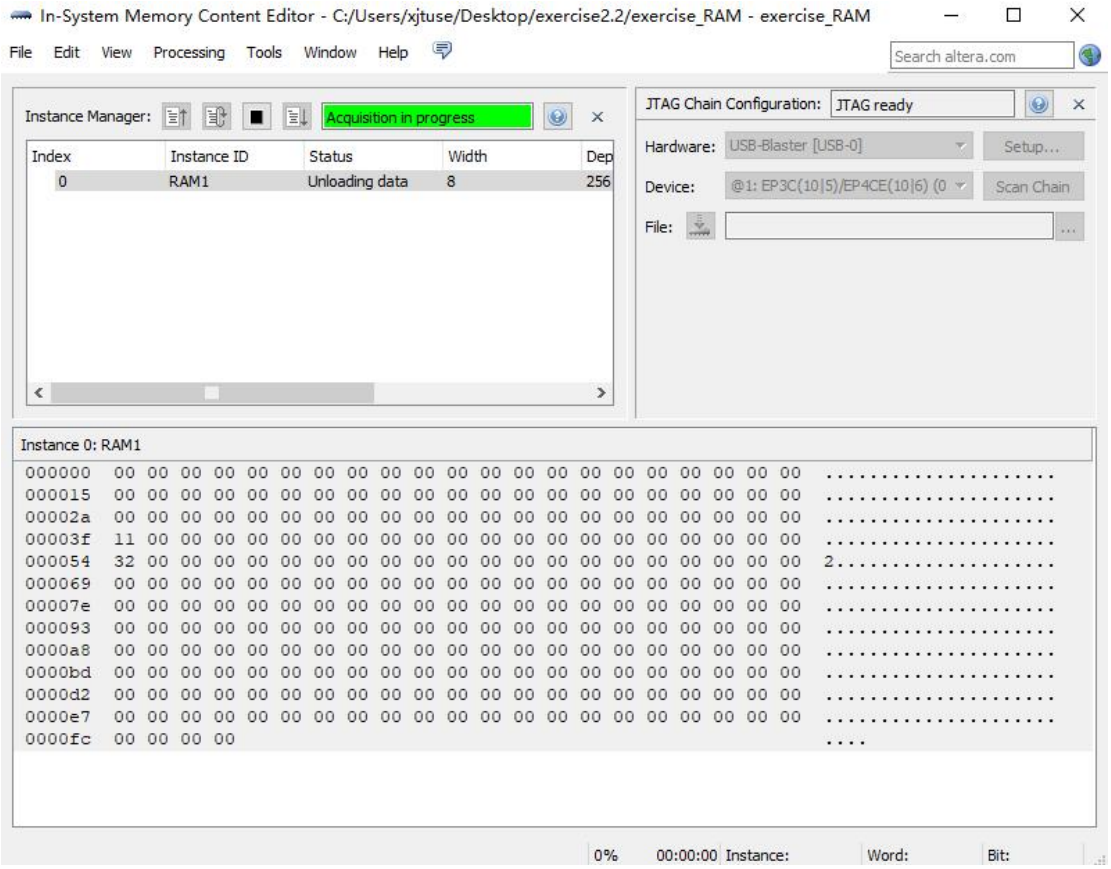
（2）





3FH 写入 11，读出 11





54H 写入 32，读出 32

八 思考题

（一）LPM_RAM_DQ 数据初始化

1. 在 Quartus II 中，可以通过 Memory Initialization File（.mif 文件）或 Hexadecimal（Intel - Format）File（.hex 文件）对 LPM_RAM_DQ 进行数据初始化。
2. 具体操作步骤如下：
    1. 在 MegaWizard Plug - In Manager 中设置 LPM_RAM 参数时，对于“Initialize memory content data”选项，选择“Yes”。
    2. 点击“Browse...”按钮，选择已创建或准备好的.mif 或.hex 文件作为初始化文件。这些文件应包含要初始化的数据，格式需符合 Quartus II 软件的要求。
    3. 确保设置初始化文件对应的端口正确，并且根据需要选择是否允许 In - System Memory Content Editor 捕获和更新内容独立于系统时钟。
    4. 完成设置后，在编译和下载过程中，初始化数据将被加载到 LPM_RAM_DQ 中。

（二）LPM_RAM_DQ 参数文件导入和存储

1. **导入参数文件**
    1. 在创建或编辑 LPM_RAM 时，如前面所述，在 MegaWizard Plug - In Manager 的相应页面中找到“Mem Init”部分。
    2. 点击“Browse...”按钮，选择要导入的参数文件（.mif 或.hex 文件）。确保文件路径正确且文件格式符合要求。
    3. 软件会自动解析文件中的参数信息，并将其应用于 LPM_RAM 的初始化设置。
2. **存储参数文件**
    1. 在 Quartus II 软件中，对于已经创建或修改好的 LPM_RAM 参数设置，如果想要保存为一个独立的参数文件（例如，以便在其他项目中复用或备份），可以在 MegaWizard Plug - In Manager 中完成参数设置后，在“Output Files”部分查看生成的相关文件。
    2. 其中包含描述 LPM_RAM 参数设置的文件（如.vhd 文件或其他相关文件），可以将这些文件复制到指定的存储位置进行保存。同时，如果在初始化过程中使用了自定义的.mif 或.hex 文件，也应妥善保存这些文件，以便在需要时再次使用。
    3. 另外，在整个工程目录中，也可以找到与 LPM_RAM 相关的文件，包括生成的配置文件、编译文件等，可以根据项目管理的需要进行备份和存储，确保参数文件和相关设置的可追溯性和可复用性。